Understanding WebBaseLoader

1. Today's Main Question

Suppose data is available on a website.

How can we:

Read Website Data

↓

Extract Text

↓

Create LangChain Document Objects

For this purpose, we can use:

WebBaseLoader

2. What Is WebBaseLoader?

Definition

WebBaseLoader is a LangChain document-loading integration that retrieves HTML web pages, extracts text from the parsed HTML, and returns Document objects.

It uses:

Web Requests

+

Beautiful Soup

for retrieving and parsing HTML content.

Simple flow:

Website URL

↓

WebBaseLoader

↓

HTML Page

↓

Extract Text

↓

LangChain Document

Important:

WebBaseLoader works with specified web-page URLs. It does not automatically load an entire website.

3. Master Formula

Web Page URL

↓

WebBaseLoader

↓

Retrieve HTML

↓

Parse HTML

↓

Extract Text

↓

Document

4. How WebBaseLoader Works Internally

The internal process can be understood in four major steps.

Step 1 — Retrieve HTML

WebBaseLoader downloads HTML content from the given website URL using an HTTP request.

URL

↓

HTTP Request

↓

HTML

Step 2 — Parse HTML

The downloaded HTML is parsed using:

Beautiful Soup

Beautiful Soup helps analyse the structure of the HTML page.

HTML

↓

Beautiful Soup

↓

Parsed HTML

Step 3 — Extract Text

WebBaseLoader extracts text from the parsed HTML.

Conceptually, it uses:

get_text()

This removes HTML tags and gives textual content.

Example:

<h1>Welcome</h1>

Learn Python

becomes text such as:

Welcome Learn Python

Step 4 — Create Document

The extracted text becomes:

document.page_content

Website information becomes:

document.metadata

Complete flow:

Website

↓

HTML

↓

Beautiful Soup

↓

Text

↓

Document

   |

   ├── page_content

   └── metadata

5. Understanding WebBaseLoader Metadata

A loaded webpage may provide metadata such as:

Metadata Field	**Purpose**
source	Original page URL
title	HTML page title
description	Available HTML meta description
language	Language declared by the page

Example:

```
{
   "source": "https://example.com/",
   "title": "Python Course",
   "description": "Python training information",
   "language": "en"
}
```

6. Metadata Memory Formula

source

↓

Which URL?

title

↓

What is the page title?

description

↓

What description is provided?

language

↓

What language is declared?

7. Important Import

```
from langchain_community.document_loaders import (
   WebBaseLoader
)
```

8. Basic Usage

```
loader = WebBaseLoader(
    web_path="https://example.com/"
)
```

Then:

```
documents = loader.load()
```

Simple:

URL

↓

WebBaseLoader

↓

load()

↓

Documents

9. Important WebBaseLoader Parameters

Parameter	Purpose
web_path	Single URL or supported sequence of URLs
web_paths	Sequence of URLs to load

header_template	Request headers including User-Agent
requests_kwargs	Request options such as timeout
raise_for_status	Raise an error for unsuccessful HTTP status codes
bs_kwargs	Options passed to Beautiful Soup while parsing
bs_get_text_kwargs	Options used while extracting text
requests_per_second	Concurrency limit for supported multiple-request operations

10. Parameter 1 — web_path

web_path identifies the webpage that we want to load.

Example:

```
loader = WebBaseLoader(
   web_path="https://example.com/"
)
```

Simple:

web_path

↓

Which Page?

11. Parameter 2 — web_paths

web_paths allows us to specify multiple pages.

Example:

```
web_paths=[
   "https://example.com/page1",
   "https://example.com/page2"
]
```

Concept:

Page 1 URL

Page 2 URL

Page 3 URL

↓

web_paths

↓

Load Multiple Pages

12. web_path vs web_paths

web_path

↓

Single Page

web_paths

↓

Multiple Pages

Easy memory:

path = One

paths = Many

13. Parameter 3 — header_template

header_template lets us provide HTTP request headers.

Example:

```
header_template={
   "User-Agent":
   "LangChainDay4EducationalDemo/1.0"
}
```

14. What Is User-Agent?

A User-Agent is an HTTP request header.

It tells the website server which application or client is sending the request.

Concept:

Python Program

↓

HTTP Request

↓

User-Agent

↓

Website Server

Example:

```
header_template={
   "User-Agent":
   "LangChainDay4EducationalDemo/1.0"
}
```

15. Parameter 4 — requests_kwargs

requests_kwargs lets us configure request behaviour.

Example:

```
requests_kwargs={
   "timeout": 15
}
```

This sets a request timeout.

Concept:

Request Sent

↓

Wait for Server

↓

Maximum 15 Seconds

This helps prevent the program from waiting indefinitely for a slow server.

16. Parameter 5 — raise_for_status

Example:

```
raise_for_status=True
```

With this setting, unsuccessful HTTP responses can be raised as exceptions.

Concept:

HTTP Response

↓

Success?

/ \

Yes No

↓ ↓

Continue Raise Error

17. Parameter 6 — bs_get_text_kwargs

This parameter controls aspects of extracting text from parsed HTML.

Example:

```
bs_get_text_kwargs={
   "separator": " ",
   "strip": True
}
```

Two important options here are:

separator

strip

18. Understanding "separator": " "

separator specifies what should be placed between extracted text pieces.

Suppose HTML is:

<h1>Welcome to DURGASOFT</h1>

Learn Python

Learn AI

Different separators produce different output.

Separator	Output
" "	Welcome to DURGASOFT Learn Python Learn AI
"\n"	Each text piece appears on a new line
","	Welcome to DURGASOFT,Learn Python,Learn AI
""	Welcome to DURGASOFTLearn PythonLearn AI

Conclusion:

separator controls how extracted text pieces are joined together.

19. Understanding "strip": True

Suppose HTML contains:

  Learn Python

Without stripping:

" Learn Python "

With:

strip=True

we get:

"Learn Python"

Conclusion:

strip=True removes unnecessary surrounding whitespace and makes extracted text cleaner.

20. separator vs strip

separator

↓

How Text Pieces

Are Joined

strip

↓

Remove Unnecessary

Surrounding Spaces

21. Basic Program — First WebBaseLoader Example

```
from langchain_community.document_loaders import (
    WebBaseLoader
)



url = (
    "https://www.durgasoftonline.com/"
    "s/pages/new-batches/"
)



loader = WebBaseLoader(
    web_path=url
)
```

```
documents = loader.load()



print(
    len(documents)
)  # 1



document = documents[0]



print(
    document.page_content
)



print(
    document.metadata
)
```

22. Understanding the Basic Program

First:

```
url = (
    "https://www.durgasoftonline.com/"
    "s/pages/new-batches/"
)
```

specifies the webpage.

Then:

```
loader = WebBaseLoader(
    web_path=url
)
```

creates the loader.

Then:

```
documents = loader.load()
```

loads the webpage into LangChain Documents.

Then:

```
document = documents[0]
```

accesses the first Document.

23. Basic Program Flow

URL

↓

WebBaseLoader

↓

loader.load()

↓

documents

↓

documents[0]

↓

Document

|

├── page_content

└── metadata

24. Program — WebBaseLoader with Important Parameters

```
from langchain_community.document_loaders import (
    WebBaseLoader
)



# ========================================
# SPECIFY WEB PAGE
# ========================================

url = (
    "https://www.durgasoftonline.com/"
    "s/pages/new-batches/"
)



# ========================================
# CREATE WEB LOADER
# ========================================
```

```
loader = WebBaseLoader(
    web_path=url,

    header_template={
        "User-Agent":
        "LangChainDay4EducationalDemo/1.0"
    },

    requests_kwargs={
        "timeout": 15
    },

    raise_for_status=True,

    bs_get_text_kwargs={
        "separator": " ",
        "strip": True
    }
)
```

```
# ========================================
# LOAD WEB PAGE
# ========================================

try:

    documents = loader.load()



    print("TOTAL DOCUMENTS")
    print("---------------")

    print(
        len(documents)
    )



    document = documents[0]
```

```
print("\nSOURCE")
print("------")

print(
    document.metadata["source"]
)


print("\nTITLE")
print("-----")

print(
    document.metadata.get(
        "title",
        "Not available"
    )
)
```

```
    print("\nPAGE CONTENT")
    print("------------")


    text = " ".join(
        document.page_content.split()
    )


    print(
        text
    )

    # print(text[:500])


except Exception as error:

    print(
        "WEB PAGE COULD NOT BE LOADED"
```

```
)


print(
    "ERROR TYPE:"
)

print(
    type(error).__name__
)


print(
    "ERROR DETAILS:"
)

print(
    error
)
```

25. Program Architecture

URL

↓

WebBaseLoader

|

├── header_template

├── requests_kwargs

├── raise_for_status

└── bs_get_text_kwargs

↓

loader.load()

↓

Document

|

├── source

├── title

└── page_content

26. Why Use try-except?

Website loading may fail.

Possible issues can include:

HTTP Error

Timeout

Connection Problem

Unavailable Page

So the program uses:

try:

and:

except Exception as error:

Simple flow:

Try to Load

↓

Success?

/ \

Yes No

↓ ↓

Use Catch

Data Error

27. Understanding Error Type

type(error).__name__

gives the type of error.

Concept:

Error Object

↓

type(error)

↓

Class

↓

.__name__

↓

Error Type Name

28. Understanding Error Details

print(

error

)

shows the error information.

So:

ERROR TYPE

↓

What kind of error?

ERROR DETAILS

↓

What exactly happened?

29. Understanding This Line

```
text = " ".join(
   document.page_content.split()
)
```

First:

document.page_content.split()

splits the text based on whitespace.

Example:

"Python

is easy"

becomes approximately:

[

```
    "Python",
    "is",
    "easy"
]
```

Then:

```
" ".join(...)
```

joins those values using one space.

Final result:

```
Python is easy
```

So this line helps create cleaner output.

30. Display Only First 500 Characters

Instead of:

```
print(
    text
)
```

we may use:

```
print(
    text[:500]
)
```

This displays only the first 500 characters.

Concept:

Very Large Web Page

↓

text[:500]

↓

First 500 Characters

31. Program — Loading Multiple URLs

```
from langchain_community.document_loaders import (
   WebBaseLoader
)



# ========================================
# SPECIFY MULTIPLE WEB PAGES
# ========================================

urls = [
   "https://www.durgasoftonline.com/",
```

```
    (
        "https://www.durgasoftonline.com/"
        "s/pages/new-batches/"
    ),

    "https://www.durgasoftonline.com/faqs"
]


# ========================================
# CREATE WEB LOADER
# ========================================

loader = WebBaseLoader(
    web_paths=urls,

    header_template={
        "User-Agent":
        "LangChainDay4EducationalDemo/1.0"
```

```
    },

    requests_kwargs={
        "timeout": 15
    },

    raise_for_status=True,

    bs_get_text_kwargs={
        "separator": " ",
        "strip": True
    }
)



# ========================================
# LOAD MULTIPLE WEB PAGES
# ========================================

try:
```

```
documents = loader.load()



print("TOTAL DOCUMENTS")
print("---------------")

print(
    len(documents)
)



# ====================================
# DISPLAY EACH DOCUMENT
# ====================================

for index, document in enumerate(
    documents,
    start=1
):
```

```
print(
    "\nDOCUMENT",
    index
)

print(
    "====================="
)


print("\nSOURCE")
print("------")

print(
    document.metadata.get(
        "source",
        "Not available"
    )
)
```

```
print("\nTITLE")
print("-----")

print(
    document.metadata.get(
        "title",
        "Not available"
    )
)


print("\nPAGE CONTENT")
print("------------")


text = " ".join(
    document.page_content.split()
)
```

```
        print(
            text[:500]
        )


except Exception as error:

    print(
        "WEB PAGES COULD NOT BE LOADED"
    )


    print(
        "ERROR TYPE:"
    )

    print(
        type(error).__name__
```

```
    )



    print(
        "ERROR DETAILS:"
    )

    print(
        error
    )
```

32. Multiple URLs Flow

URL 1

URL 2

URL 3

↓

web_paths

↓

WebBaseLoader

↓

loader.load()

↓

Documents

↓

Loop Through Documents

33. Why Use enumerate()?

We use:

```
for index, document in enumerate(
    documents,
    start=1
):
```

This gives:

1 → First Document

2 → Second Document

3 → Third Document

Because:

start=1

starts displayed numbering from 1.

34. Accessing Metadata Safely

Instead of:

```
document.metadata["title"]
```

we may use:

```
document.metadata.get(
   "title",
   "Not available"
)
```

Concept:

```
title Available?
   / \
 Yes  No
 ↓    ↓
Title  "Not available"
```

35. Single URL vs Multiple URLs

Single URL

```
web_path=url
```

Flow:

One URL

↓

One Page Loading

Multiple URLs

web_paths=urls

Flow:

URL 1

URL 2

URL 3

↓

Multiple Page Loading

36. Program — Local Demonstration Website

This example creates a local webpage and loads it using WebBaseLoader.

It helps us understand WebBaseLoader without depending on an external website.

```
from http.server import (
    BaseHTTPRequestHandler,
    ThreadingHTTPServer
)
```

```
from threading import Thread

from langchain_community.document_loaders import (
    WebBaseLoader
)



# ========================================
# CREATE A LOCAL DEMO WEB PAGE
# ========================================

html_content = """
<html lang="en">

<head>

    <title>
        Institute Notices
    </title>
```

```
    <meta
      name="description"
      content="Training institute updates"
    >

</head>

<body>

    <main>

      <h1>
        Institute Notices
      </h1>

      <p>
        Python batch starts on Monday.
      </p>

      <p>
```

```
            Library opens at 8 AM.
        </p>

    </main>

</body>

</html>
"""



# ========================================
# DEFINE DEMO HTTP SERVER
# ========================================

class DemoHandler(
    BaseHTTPRequestHandler
):

    def do_GET(self):
```

```
body = html_content.encode(
    "utf-8"
)


self.send_response(
    200
)


self.send_header(
    "Content-Type",
    "text/html; charset=utf-8"
)


self.send_header(
    "Content-Length",
    str(len(body))
```

```
        )


        self.end_headers()


        self.wfile.write(
            body
        )


    def log_message(
        self,
        *args
    ):

        pass


# ========================================
```

```
# START LOCAL WEB SERVER
# ======================================

server = ThreadingHTTPServer(
    (
        "127.0.0.1",
        0
    ),

    DemoHandler
)


thread = Thread(
    target=server.serve_forever,
    daemon=True
)


thread.start()
```

```
try:

    # ====================================
    # CREATE WEB LOADER
    # ====================================

    url = (
        f"http://127.0.0.1:"
        f"{server.server_port}/"
    )


    loader = WebBaseLoader(
        web_path=url,

        header_template={
            "User-Agent":
            "LangChainDay4LocalDemo/1.0"
```

```
    },

    bs_get_text_kwargs={
        "separator": " ",
        "strip": True
    },

    requests_kwargs={
        "timeout": 10
    },

    raise_for_status=True
)



# ====================================
# LOAD LOCAL WEB PAGE
# ====================================

documents = loader.load()
```

```
document = documents[0]


# ====================================
# DISPLAY RESULTS
# ====================================

print(
    "TOTAL DOCUMENTS:",
    len(documents)
)


print(
    "\nTITLE:"
)

print(
```

```
    document.metadata.get(
        "title",
        "Not available"
    )
)


print(
    "\nDESCRIPTION:"
)

print(
    document.metadata.get(
        "description",
        "Not available"
    )
)


print(
```

```
    "\nLANGUAGE:"
)

print(
    document.metadata.get(
        "language",
        "Not available"
    )
)


print(
    "\nSOURCE IS LOCAL:"
)

print(
    document.metadata["source"].startswith(
        "http://127.0.0.1:"
    )
)
```

```
    print(
        "\nPAGE CONTENT:"
    )

    print(
        " ".join(
            document.page_content.split()
        )
    )


finally:

    server.shutdown()

    server.server_close()
```

37. Why Create a Local Demo Website?

The local example allows us to understand the complete process:

Create HTML

↓

Start HTTP Server

↓

Create Local URL

↓

WebBaseLoader

↓

Load Page

↓

Document

38. Understanding html_content

The variable:

html_content

contains a simple HTML page.

Important elements include:

<title>Institute Notices</title>

This may become:

metadata["title"]

The meta description:

```
<meta
   name="description"
   content="Training institute updates"
>
```

may become:

metadata["description"]

The HTML language:

```
<html lang="en">
```

may become:

metadata["language"]

39. Understanding the Page Body

The page contains:

```
<h1>Institute Notices</h1>
```

   Python batch starts on Monday.

   Library opens at 8 AM.

This textual information becomes part of:

document.page_content

40. Local Server Architecture

html_content

   ↓

DemoHandler

   ↓

ThreadingHTTPServer

   ↓

Local URL

   ↓

WebBaseLoader

↓

Document

41. Understanding 127.0.0.1

The local URL is created using:

127.0.0.1

Example:

http://127.0.0.1:<port>/

The program checks:

```
document.metadata["source"].startswith(
    "http://127.0.0.1:"
)
```

This verifies that the source URL is the local server.

42. Understanding Dynamic Port

The server is created with:

("127.0.0.1", 0)

Then the program gets the selected port using:

server.server_port

and creates:

```
url = (
    f"http://127.0.0.1:"
    f"{server.server_port}/"
)
```

43. Why Use a Thread?

The local HTTP server runs using:

```
thread = Thread(
    target=server.serve_forever,
    daemon=True
)
```

Then:

```
thread.start()
```

This starts the server while allowing the remaining Python program to continue.

44. Why Use finally?

The program uses:

```
finally:
```

to stop and close the local server.

server.shutdown()

server.server_close()

Concept:

Start Server

↓

Run Program

↓

Success or Error

↓

finally

↓

Stop Server

45. Metadata in Local Demo

The program reads:

document.metadata.get(

"title",

"Not available"

)

document.metadata.get(

```
    "description",
    "Not available"
)
document.metadata.get(
    "language",
    "Not available"
)
```

and:

```
document.metadata["source"]
```

So the metadata may help us understand:

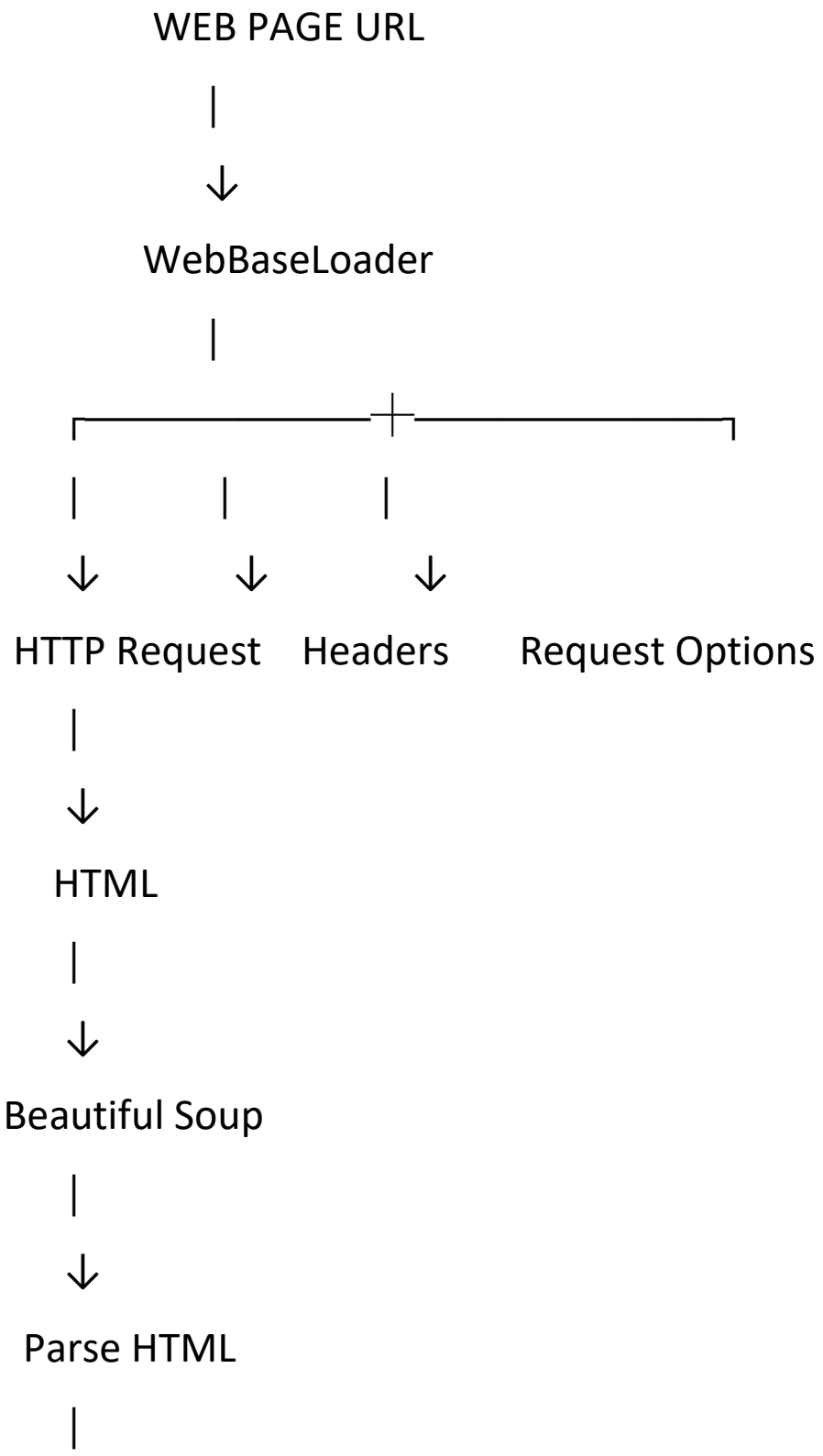
Title

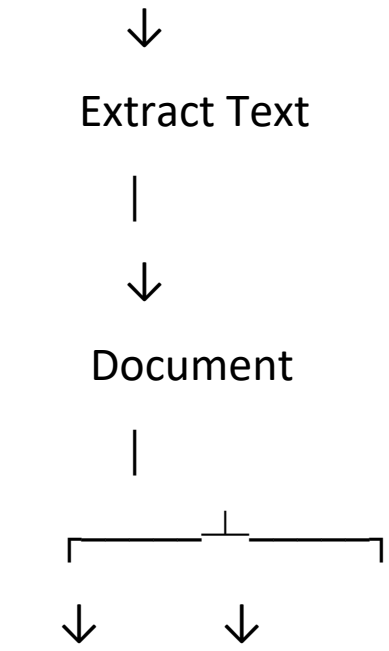
Description

Language

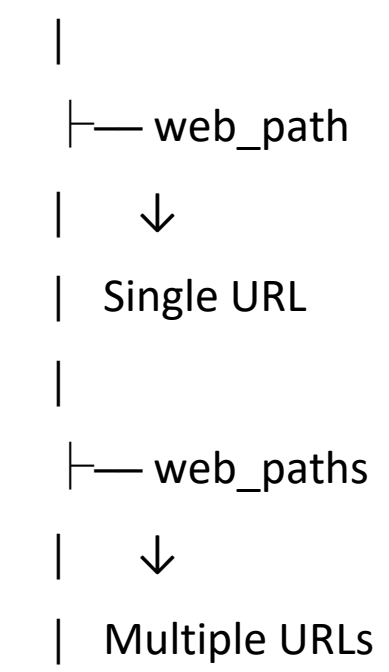
Source

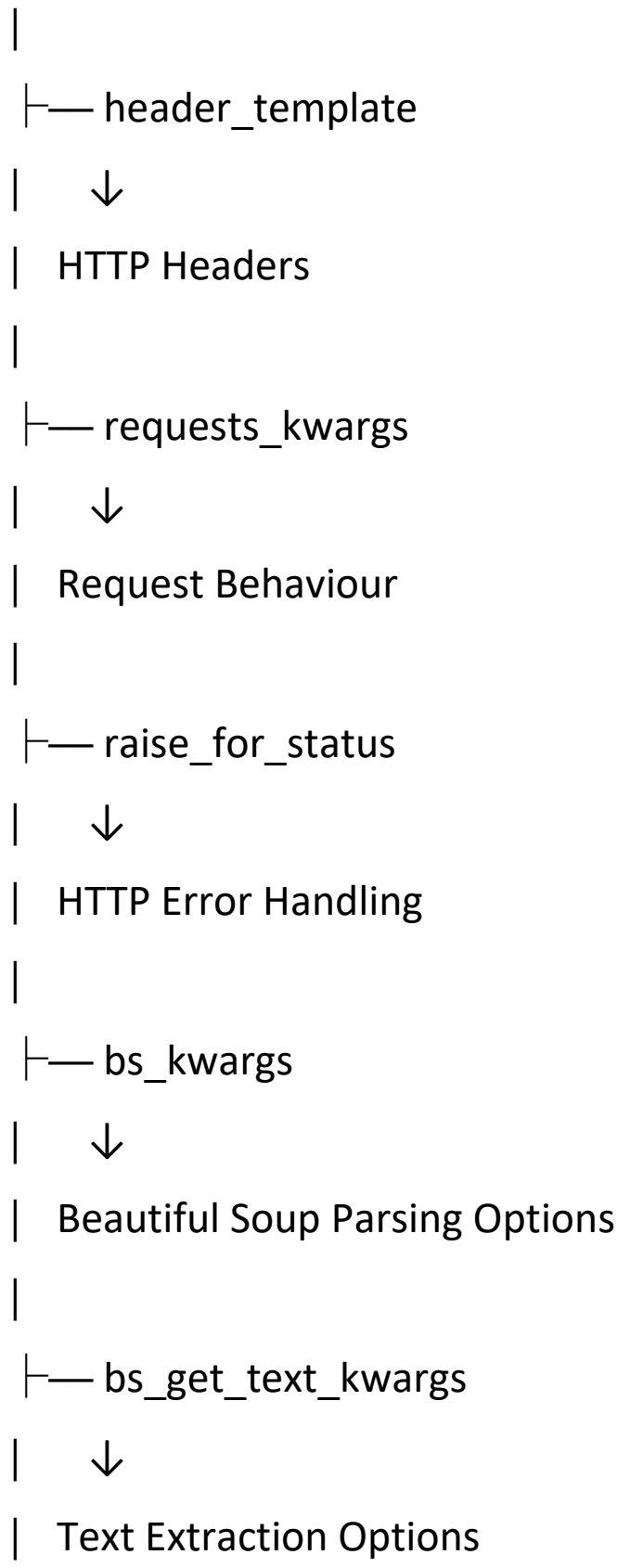
46. Complete WebBaseLoader Architecture

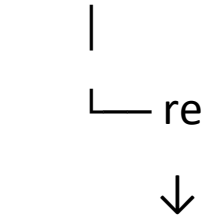




47. Important Parameter Architecture




├── header_template
↓
HTTP Headers
├── requests_kwargs
↓
Request Behaviour
├── raise_for_status
↓
HTTP Error Handling
├── bs_kwargs
↓
Beautiful Soup Parsing Options
├── bs_get_text_kwargs
↓
Text Extraction Options



48. Important Syntax Revision

Import

```
from langchain_community.document_loaders import (
   WebBaseLoader
)
```

Single URL

```
loader = WebBaseLoader(
   web_path=url
)
```

Multiple URLs

```
loader = WebBaseLoader(
    web_paths=urls
)
```

User-Agent

```
header_template={
    "User-Agent":
    "LangChainDay4EducationalDemo/1.0"
}
```

Timeout

```
requests_kwargs={
    "timeout": 15
}
```

Raise HTTP Errors

```
raise_for_status=True
```

Clean Text Extraction

```
bs_get_text_kwargs={
   "separator": " ",
   "strip": True
}
```

Load

```
documents = loader.load()
```

Page Content

```
document.page_content
```

Metadata

```
document.metadata
```

Source

```
document.metadata["source"]
```

Title

```
document.metadata.get(
   "title",
   "Not available"
)
```

49. Common Mistake — Thinking WebBaseLoader Loads the Entire Website Automatically

Wrong:

Website Domain

↓

Automatically Load Every Page

The source notes state that WebBaseLoader works with specified webpage URLs.

So conceptually:

Specified URL

↓

Load That Page

50. Common Mistake — Confusing web_path and web_paths

web_path

↓

Single URL

web_paths

↓

Multiple URLs

51. Common Mistake — Thinking HTML Tags Become the Main Output

The objective is to extract text from the parsed HTML.

Example:

<h1>Python</h1>

becomes:

Python

Concept:

HTML Tags

↓

Parsing

↓

Text

52. Common Mistake — Ignoring Request Errors

Website loading can fail.

That is why a program may use:

try:

except Exception as error:

and:

raise_for_status=True

53. Common Mistake — Ignoring Whitespace

Webpage text may contain irregular whitespace.

We can clean displayed text using:

```
text = " ".join(
   document.page_content.split()
)
```

54. Common Mistake — Assuming Metadata Is Always Available

Instead of always using:

document.metadata["title"]

the examples use:

```
document.metadata.get(
   "title",
   "Not available"
)
```

when a metadata value may not be present.

55. Limitation of WebBaseLoader

A very important limitation from the source notes:

WebBaseLoader cannot execute JavaScript.

It can load static HTML content, but it may miss content generated dynamically using JavaScript.

Concept:

Static HTML Content

↓

WebBaseLoader

↓

Can Read

But:

JavaScript-Generated Content

↓

WebBaseLoader

↓

May Miss It

56. Alternatives for JavaScript-Rendered Websites

The source notes mention two alternatives:

1. PlaywrightURLLoader

↓

Recommended

2. SeleniumURLLoader

↓

Another Alternative

57. Static vs JavaScript-Rendered Website

STATIC PAGE

Server sends useful HTML

↓

WebBaseLoader

↓

Extract Text

DYNAMIC PAGE

Initial HTML

↓

JavaScript Executes

↓

Content Appears

↓

WebBaseLoader may miss

that generated content

58. TextLoader vs PyPDFLoader vs CSVLoader vs WebBaseLoader

Loader	Source
TextLoader	Text file
PyPDFLoader	PDF file
CSVLoader	CSV file
WebBaseLoader	HTML webpage

Easy memory:

TXT

↓

TextLoader

PDF

↓

PyPDFLoader

CSV

↓

CSVLoader

WEB

↓

WebBaseLoader

59. Important WebBaseLoader Revision Points

1. WebBaseLoader loads HTML web pages.
2. It retrieves HTML using web requests.
3. It uses Beautiful Soup to parse HTML.
4. Extracted text becomes page_content.
5. Website details become metadata.
6. source normally identifies the original URL.
7. title may contain the HTML page title.
8. description may contain the HTML meta description.
9. language may contain the declared page language.
10. web_path identifies a page to load.
11. web_paths can be used for multiple URLs.
12. header_template supplies HTTP headers.
13. User-Agent identifies the requesting client.
14. requests_kwargs controls request behaviour such as timeout.
15. raise_for_status=True helps surface unsuccessful HTTP responses as exceptions.
16. bs_get_text_kwargs controls text extraction behaviour.
17. separator determines how text pieces are joined.

18. strip=True removes surrounding whitespace.
19. try-except can be used for webpage-loading errors.
20. WebBaseLoader does not execute JavaScript according to the source notes.

60. Four-Step Memory Formula

1. RETRIEVE

↓

Get HTML

2. PARSE

↓

Beautiful Soup

3. EXTRACT

↓

Get Text

4. CREATE

↓

Document

Easy memory:

Retrieve

↓

Parse

↓

Extract

↓

Document

61. Final Master Diagram

WEBSITE

|

↓

URL

|

↓

WebBaseLoader

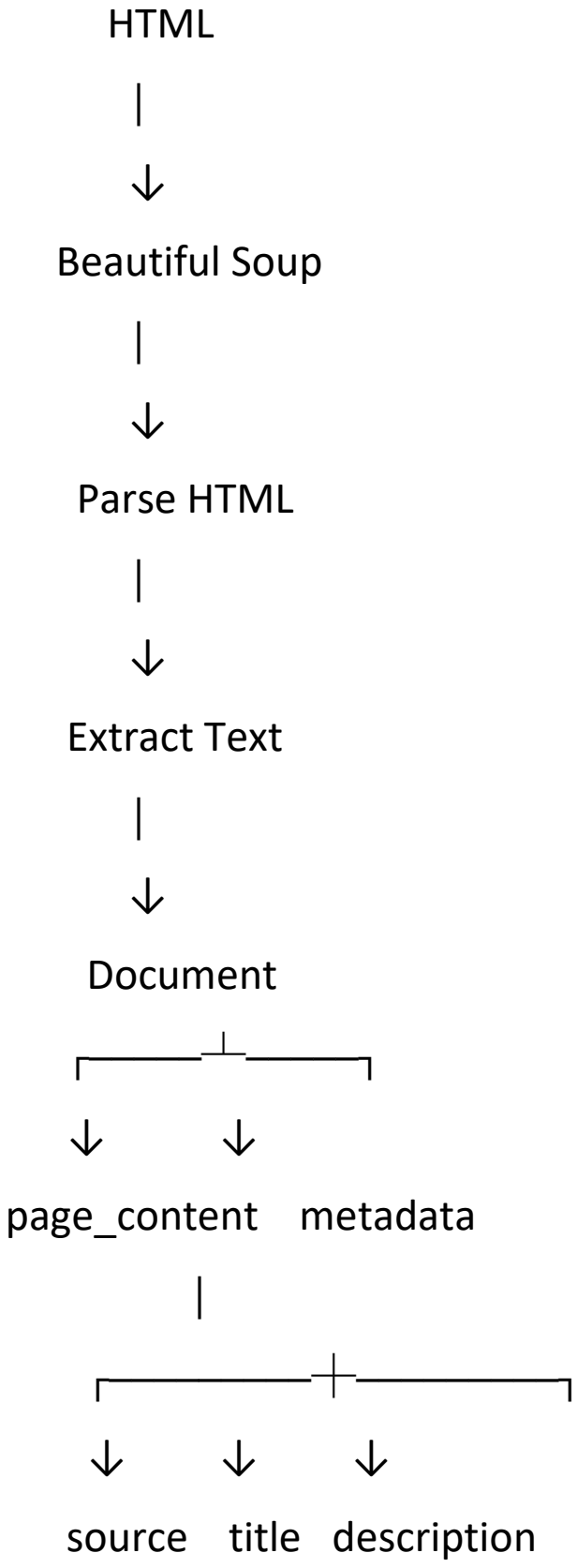
|

↓

HTTP Request

|

↓


HTML
Beautiful Soup
Parse HTML
Extract Text
Document
page_content
metadata
source
title
description

+

language

62. One-Line Conclusion

WebBaseLoader retrieves a specified HTML webpage, parses it using Beautiful Soup, extracts available text, and converts that information into LangChain Document objects containing page_content and webpage metadata.